

JSON Web Tokens (JWT)

Security Notes — Knowledge Base

1. What is a JWT?

A JSON Web Token is a compact, URL-safe, self-contained token format used to represent claims securely between two parties. It's commonly used for authentication and information exchange in web APIs.

2. Structure

A JWT consists of three Base64URL-encoded parts separated by dots: Header.Payload.Signature.

Structure Breakdown

Part	Contents
Header	Token type (JWT) and signing algorithm (e.g., HS256, RS256)
Payload	Claims — statements about the user/entity (e.g., sub, exp, roles)
Signature	Cryptographic signature verifying the token wasn't tampered with

3. Example (decoded)

```
// Header
{ "alg": "HS256", "typ": "JWT" }

// Payload
{ "sub": "user123", "role": "admin", "exp": 1735689600 }

// Signature = HMACSHA256(base64(header) + "." + base64(payload), secret)
```

4. How Authentication Flow Works

- User logs in with credentials; server verifies them and issues a signed JWT.
- Client stores the token (e.g., in memory or an HttpOnly cookie) and sends it in the Authorization header on subsequent requests.
- Server verifies the signature on each request — no database lookup needed, since the token is self-contained (stateless auth).
- Token expires (exp claim) and the client must re-authenticate or use a refresh token.

5. Important Security Practices

- Always set a short expiration (exp) and use refresh tokens for long sessions.

- Never store sensitive secrets in the payload — it's only Base64-encoded, not encrypted, and can be read by anyone.
- Use strong algorithms (RS256/ES256 for asymmetric signing) and never accept 'alg: none'.
- Store tokens securely on the client (HttpOnly, Secure cookies are safer than localStorage against XSS).
- Implement token revocation strategy (e.g., short-lived tokens + refresh token rotation) since JWTs can't be easily invalidated before expiry.

6. JWT vs Session Cookies

Aspect	JWT (stateless)	Session Cookies (stateful)
Server storage	None needed	Session store required
Revocation	Harder (needs extra logic)	Easy (delete session)
Scalability	Easier across distributed servers	Needs shared session store